FlowSense

Sistema de análisis de flujo peatonal para fijación de precios de arriendo
en espacios comerciales mediante visión artificial

Python 3.11 · YOLOv8 · Spring Boot 3 · React 18 · MySQL

Sigla	Asignatura	Tiempo	Equipos
TPY 1101	Taller Aplicado de Programación	Semestre	3 integrantes

Versión 2.0 — Taller Aplicado de Programación TPY1101

1. Antecedentes Personales

Campo	Integrante 1	Integrante 2	Integrante 3
RUT	20033556-2	*[Por completar]*	*[Por completar]*
Nombres	*Fernando Andres*	*[Por completar]*	*[Por completar]*
Apellidos	*Huamanchumo Zuñiga*	*[Por completar]*	*[Por completar]*
Rol	*Front-end / React / Dashboard*	*Back-end / Python / YOLOv8 / Spring Boot*	*DBA / MySQL / QA / Documentación*

2. Descripción del Proyecto

Nombre	**FlowSense — Sistema de análisis de flujo peatonal para fijación de precios de arriendo en espacios comerciales mediante visión artificial**
Descripción	Plataforma web donde el administrador de un mall, galería comercial o feria sube un video MP4 de sus pasillos, define zonas sobre el plano del recinto y obtiene automáticamente métricas de flujo peatonal diferenciadas por zona y franja horaria. El módulo de detección usa Python con YOLOv8 preentrenado; el backend en Spring Boot con Java; el frontend en React con librerías especializadas (heatmap.js, react-konva, recharts, jsPDF); y la persistencia en MySQL.
Problemática	Los administradores de malls, galerías comerciales y ferias comerciales fijan precios de arriendo sin datos objetivos de tráfico peatonal diferenciado por zona. Las soluciones de analítica de tráfico (RetailNext, FootfallCam, ShopperTrak) requieren entre $2.000 y $50.000 USD en hardware especializado, siendo inaccesibles para operadores de espacios medianos. No existe solución accesible basada en visión artificial orientada a administradores de espacios comerciales en Chile.
Solución esperada	Aplicación web donde el administrador: (1) sube video MP4 del recinto, (2) define zonas sobre el plano (entrada, pasillo A, pasillo B, zona premium), (3) recibe mapa de calor con colores rojo-azul según densidad peatonal, y (4) consulta métricas comparativas por zona: personas detectadas, porcentaje del total, zona de mayor y menor afluencia. Sin hardware adicional. Reporte PDF descargable para justificar precios ante arrendatarios.
Objetivo general	Desarrollar un sistema web de análisis de flujo peatonal mediante visión artificial que permita a administradores de espacios comerciales aforar el tráfico diferenciado por zona, apoyando la fijación objetiva de precios de arriendo en malls, galerías y ferias comerciales.
Objetivo específico 1	Implementar un módulo de detección y conteo de personas en video mediante YOLOv8 que genere métricas de flujo peatonal segmentadas por zona geográfica y franja horaria.
Objetivo específico 2	Construir un dashboard web que presente las métricas de tráfico en formato de reporte comparativo entre zonas, permitiendo al administrador visualizar diferencias de aforo que justifiquen la estructura de precios de arriendo.
Alcance MVP	INCLUYE: subida de video MP4, extracción 1 frame/segundo, detección YOLOv8 sin GPU, definición de zonas sobre plano, mapa de calor canvas, dashboard comparativo, reporte PDF, alertas de zona fría, gestión multi-recinto, configuración de umbral de confianza. NO INCLUYE: video en tiempo real, reconocimiento facial, app móvil, módulo de pagos, integración cámaras IP.

3. Contexto del Proyecto

3.1 Antecedentes y motivación

La fijación de precios de arriendo en espacios comerciales con múltiples locales o stands es una disciplina con escasa evidencia objetiva en el segmento medio del mercado chileno. Administradores de malls, galerías y ferias determinan el precio de cada local basándose en metros cuadrados, piso o percepción histórica del tráfico. Esta aproximación genera ingresos subóptimos y conflictos recurrentes con arrendatarios que cuestionan las diferencias de precio sin evidencia que las respalde.

Las soluciones de analítica de tráfico consolidadas —RetailNext, FootfallCam, ShopperTrak— están diseñadas para grandes cadenas con presupuestos de $2.000 a $50.000 USD en hardware especializado más contratos anuales. La disponibilidad de YOLOv8n como modelo preentrenado de uso libre ejecutable en CPU convencional elimina por primera vez esta barrera económica, haciendo viable una solución accesible para administradores de espacios medianos.

FlowSense desarrolla esa oportunidad usando como escenario video grabado directamente en el recinto real con cámara convencional o celular del administrador. No requiere hardware adicional ni instalación de software por parte del usuario final.

3.2 Dimensión del mercado objetivo

Segmento	Descripción	Relevancia para FlowSense
Malls y centros comerciales	Chile cerró 2024 con 264 centros comerciales operativos y 6,56 millones de m² de área bruta arrendable según la Cámara Chilena de Centros Comerciales.	El precio de cada local depende del tráfico de su pasillo. Sin datos, la negociación con arrendatarios es a ciegas. FlowSense provee la evidencia objetiva que falta.
Galerías comerciales urbanas	Cientos de galerías en Santiago y regiones con 20 a 100 locales internos, administradas por propietarios individuales o empresas familiares.	Sin acceso a sistemas de analítica. Fijan precios por metro cuadrado o tradición histórica. Máximo potencial de adopción del plan básico.
Ferias comerciales	Más de 221 ferias organizadas en Chile en 64 sectores y 25 ciudades. Solo en Santiago, recintos como Espacio Riesco albergan ferias con cientos de stands.	El organizador cobra igual por stands en zonas de distinto tráfico. FlowSense permite cobrar diferencial en la siguiente edición con respaldo de datos.

3.3 Estado del arte — Tabla de homologación

Atributo	FlowSense	RetailNext	FootfallCam	Placer.ai	Conteo manual
Modelo de negocio	SaaS web	Hardware + SaaS	Hardware dedicado	SaaS (GPS móvil)	Sin tecnología
Requiere hardware adicional	No — cámara o celular	Sí — cámaras propias	Sí — sensores techo	No — datos GPS	No
Analítica por zona interna	Sí — definida por usuario	Sí	Sí	No (solo exterior)	No
Orientado a administradores de arriendo	Sí — propósito principal	No	No	No	N/A
Reporte comparativo de zonas	Sí — PDF descargable	Sí	Sí	No	No

Atributo	FlowSense	RetailNext	FootfallCam	Placer.ai	Conteo manual
Costo estimado	Accesible / académico	$5.000–50.000 USD	$2.000–20.000 USD	$500+/mes	$0 (impreciso)
Sin identificación facial	Sí — solo bounding box	Variable	Variable	Anónimo (GPS)	Sí

3.4 Oportunidad tecnológica

Modelo YOLOv8n	Ultralytics 2023. Preentrenado con la clase 'person' del dataset COCO. Precisión >85% en interiores comerciales en CPU convencional. Licencia AGPL-3.0 libre para proyectos académicos.
Escenario de validación	Video grabado directamente en el recinto real (mall, galería o feria) con cámara del equipo. Se definen zonas sobre el plano real del espacio. Los resultados reflejan tráfico auténtico, no simulado.
Alineación con el ramo	RA1: problema real con mercado de 264 malls + cientos de galerías + 221 ferias. RA2: solución innovadora con ética documentada (sin identificación facial). RA3: validación con video real del sistema funcionando extremo a extremo.

3.5 Proceso de negocio afectado

El proceso de negocio que FlowSense transforma es la fijación de precios de arriendo y la negociación con arrendatarios. Hoy ese proceso es: administrador observa el recinto intuitivamente → estima cuáles zonas tienen más tráfico → fija precios por metro cuadrado y piso → negocia sin evidencia cuando el arrendatario cuestiona el precio → no puede demostrar que la zona premium vale más.

Con FlowSense: administrador graba video del recinto → sube el video a la plataforma → define zonas sobre el plano → obtiene métricas objetivas de tráfico por zona → fija precios proporcionales al tráfico real → negocia con datos que respaldan cada precio → compara ediciones o períodos sucesivos para detectar cambios de flujo.

3.6 Análisis de causa-efecto (Ishikawa)

CAUSAS del problema	EFECTOS del problema
Causa 1 — Sin herramientas accesibles RetailNext y FootfallCam requieren hardware de $2.000 a $50.000 USD, inaccesible para administradores de galerías y ferias. **Causa 2 — Datos de flujo no granulares** Los malls grandes tienen contadores en accesos generales pero sin granularidad por pasillo ni por local individual. **Causa 3 — Fijación de precios subjetiva** Los precios se determinan por metro cuadrado, piso y tradición histórica, sin proceso estandarizado basado en tráfico real. **Causa 4 — Sin evidencia post-evento** En ferias, el organizador no tiene datos de personas por stand para justificar precios ni ajustar la próxima edición.	**Efecto 1 — Ingresos de arriendo subóptimos** El administrador no puede demostrar objetivamente por qué el pasillo A vale el doble que el pasillo B. **Efecto 2 — Conflictos recurrentes con arrendatarios** Sin datos, los arrendatarios cuestionan diferencias de precio generando negociaciones largas y contratos por debajo del valor de mercado. **Efecto 3 — Mala asignación de rubros por zona** Sin saber qué zonas tienen mayor afluencia, el administrador no puede orientar estratégicamente qué tipo de negocio ubica en cada pasillo. **Efecto 4 — Vacancia en zonas de bajo tráfico** Los pasillos de menor circulación son más difíciles de arrendar porque no hay forma de demostrar que el precio está ajustado al tráfico real.

4. Metodología — Scrum con ciclo de vida iterativo-incremental

4.1 Marco metodológico

Se adopta Scrum como metodología de desarrollo bajo un ciclo de vida iterativo-incremental. El ciclo de vida iterativo-incremental se justifica con la naturaleza del proyecto: el sistema es funcional y evaluable desde el Sprint 2 con funcionalidad básica (subida de video + detección + conteo), y cada sprint agrega capacidades verificables sobre una base estable. Esto permite validar la solución técnica tempranamente y corregir el rumbo antes de invertir en funcionalidades avanzadas.

Scrum se elige por tres razones específicas: (1) permite entregar valor incremental — el sistema es evaluable por el docente desde el Sprint 2, no al final del semestre; (2) los roles y ceremonias generan trazabilidad del proceso, cumpliendo el requisito de documentación del ramo; (3) el tablero de Product Backlog en GitHub Projects permite al docente verificar el avance en cualquier momento.

La adaptación al equipo considera disponibilidad real de aproximadamente 9 horas semanales por integrante. Las ceremonias se realizan en formato lightweight: Standups por mensaje de texto, Reviews en videollamada de 20 minutos.

4.2 Sprints del proyecto

Fase / Sprint	Objetivo del sprint	Entregable verificable
Sprint 1 (S1-S2) Inicio	Setup del entorno, primera detección YOLOv8 funcionando, modelo MySQL creado, wireframes Figma, grabación de video demo en recinto real (mall, galería o feria).	YOLOv8 detecta personas en frame del video real + tablas MySQL creadas + prototipo Figma aprobado.
Sprint 2 (S3-S4) Pipeline core	Script Python procesa video completo y escribe detecciones en MySQL. Spring Boot orquesta el proceso. React sube video y consulta estado.	Video real subido → script Python ejecutado por Spring Boot → coordenadas en MySQL.
Sprint 3 (S5-S6) Frontend visual	Canvas heatmap en React mostrando detecciones reales. Herramienta de zonas sobre el plano del recinto funcionando.	Mapa de calor visible en browser con datos reales de MySQL y zonas configurables.
Sprint 4 (S7-S8) Integración y validación	Sistema completo integrado. Nuevas funcionalidades: exportación PDF, alertas de zona fría, multi-recinto, configuración de umbral. Pruebas E2E.	Flujo completo funcionando + funcionalidades de administrador operativas.
Sprint 5 (S9) Cierre	Dashboard métricas, ajustes finales, documentación, video demo, presentación.	Informe final + video demo de 3 min + presentación lista para defensa.

4.3 Roles Scrum del equipo

Rol Scrum	Responsable	Tecnologías principales	Responsabilidades
Product Owner	Integrante 3	MySQL, documentación, GitHub Projects	Define prioridades del backlog. Valida que cada entregable cumple el criterio de aceptación. Gestiona el tablero GitHub Projects.
Scrum Master	Integrante 2	Python, Spring Boot, Railway	Facilita las ceremonias. Identifica bloqueos técnicos. Asegura que

Rol Scrum	Responsable	Tecnologías principales	Responsabilidades
			el equipo avance según la planificación semanal.
Developer	Integrante 1	React, heatmap.js, react-konva, recharts	Desarrolla el frontend React. Participa en todas las ceremonias. Reporta avances diarios al Scrum Master vía mensaje en grupo.

4.4 Ceremonias Scrum adaptadas

Ceremonia	Adaptación al equipo
Sprint Planning (inicio de sprint)	Reunión de 30 min al inicio de cada sprint. Se seleccionan historias del backlog, se dividen en tareas y se asignan con puntos de historia (1=fácil, 2=medio, 3=difícil).
Daily Standup (diario)	Mensaje de texto diario en grupo. Formato: Hice: [X]. Haré: [Y]. Bloqueo: [Z o ninguno]. Si hay bloqueo, el Scrum Master coordina solución ese mismo día.
Sprint Review (fin de sprint)	Demo del entregable funcionando. 20-30 min. Se graba para el informe final.
Sprint Retrospectiva	15 min post-review. ¿Qué salió bien? ¿Qué mejorar? ¿Qué hacer diferente? Se registra en documento compartido en GitHub.
Backlog Refinement	30 min a mitad de cada sprint. Se revisa el backlog siguiente y se estiman tareas.

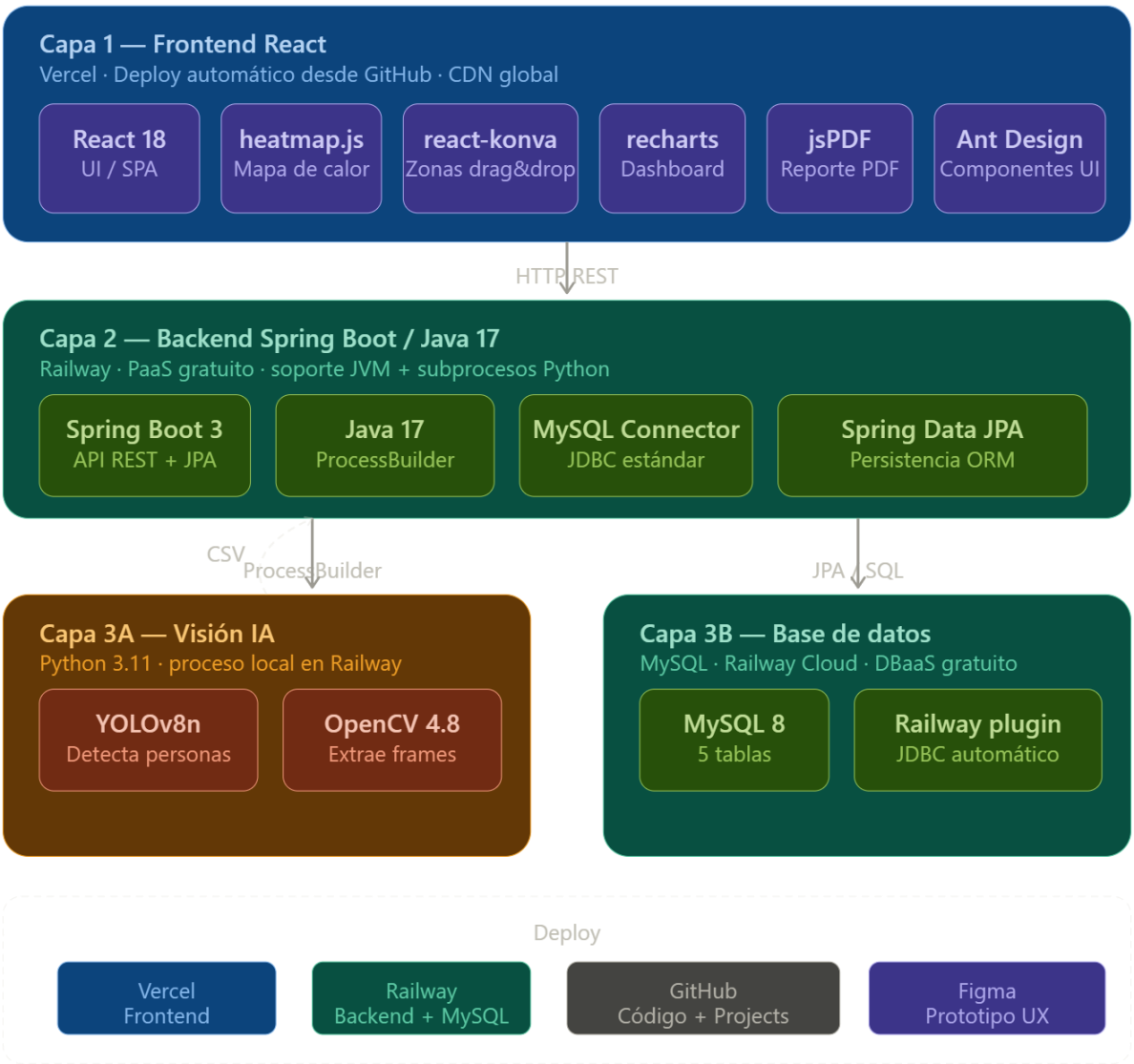
5. Descripción Técnica de la Solución

5.1 Stack tecnológico

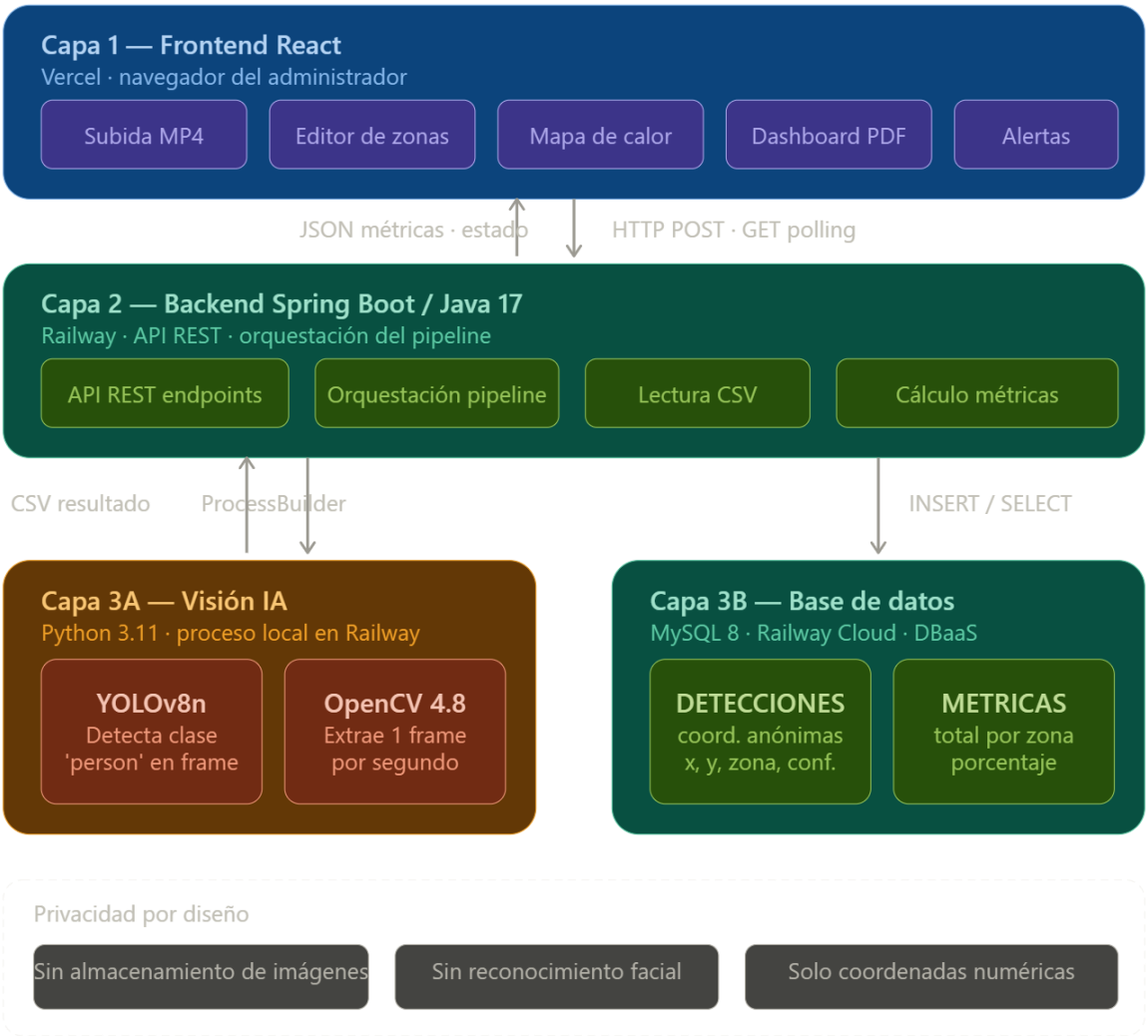
Componente	Especificación técnica	Justificación
Infraestructura cloud	Frontend React en Vercel (PaaS, gratuito, CDN global). Backend Spring Boot en Railway (PaaS, gratuito tier, soporte JVM + subprocesos Python). MySQL en Railway como plugin (DBaaS, JDBC estándar).	Sin costo de infraestructura. Railway permite ejecutar subprocesos Python en el mismo contenedor que Spring Boot, simplificando la arquitectura.
Arquitectura	4 capas: (1) Script Python (detección), (2) API REST Spring Boot (orquestación), (3) Frontend React (dashboard), (4) MySQL (persistencia). Python escribe resultados en CSV local; Spring Boot lee el CSV e inserta en MySQL.	Separación de responsabilidades. El patrón 'script invocation' elimina la complejidad de drivers de base de datos en Python y permite escalar cada capa independientemente.
Lenguajes	Python 3.11 (detección YOLOv8, extracción frames, escritura CSV). Java 17 con Spring Boot 3 (API REST, orquestación, MySQL). JavaScript ES2023 con React 18 (frontend, dashboard).	Cada lenguaje en su dominio de mayor fortaleza. Sin mezcla de responsabilidades entre capas.
Frontend React	heatmap.js (mapa de calor gradiente automático) · react-konva (canvas interactivo zonas drag & drop) · recharts (gráficos dashboard) · jsPDF + html2canvas (exportación PDF) · react-toastify (alertas zona fría) · Ant Design (UI components) · TanStack Table (tabla métricas).	Librerías especializadas que resuelven problemas complejos en pocas líneas, reduciendo la superficie de código a mantener.
Backend + Python	Spring Boot: Spring Web, Spring Data JPA, MySQL Connector/J 8. Python: ultralytics==8.x (YOLOv8), opencv-python==4.8.x (lectura video). El PDF se genera en frontend (jsPDF), eliminando dependencia de iText/JasperReports.	Stack estándar empresarial con soporte activo. Versiones fijadas en requirements.txt y pom.xml para reproducibilidad.
Base de datos MySQL	5 tablas: RECINTOS (id, nombre, imagen_plano_base64), ZONAS (id, id_recinto, nombre, x, y, ancho, alto), VIDEOS (id, id_recinto, nombre, ruta, estado, fecha_subida), DETECCIONES (id, id_video, frame_numero, zona_id, x_norm, y_norm, confianza), METRICAS (id, id_video, id_zona, total_detecciones, porcentaje_total).	Cambio desde Oracle SQL a MySQL por accesibilidad y costo cero en Railway. Compatible con Spring Data JPA sin cambios de código.
Video de validación	Video grabado directamente en recinto real (mall, galería o feria) con cámara o celular del equipo. Ángulo elevado o cenital para visibilidad de personas. Se graba en Sprint 1.	Requerimiento crítico del nuevo enfoque: el cliente es el administrador del espacio, no el dueño de la tienda. El video real valida que el sistema funciona en el contexto correcto.

Componente	Especificación técnica	Justificación
Control de versiones	GitHub. Ramas: main (producción), develop (integración), feature/python-yolo, feature/spring-api, feature/react-dashboard. Pull Request obligatorio para merge a develop. GitHub Projects para tablero Scrum.	Trazabilidad completa del trabajo por sprint. El docente puede verificar actividad continua en el historial de commits.

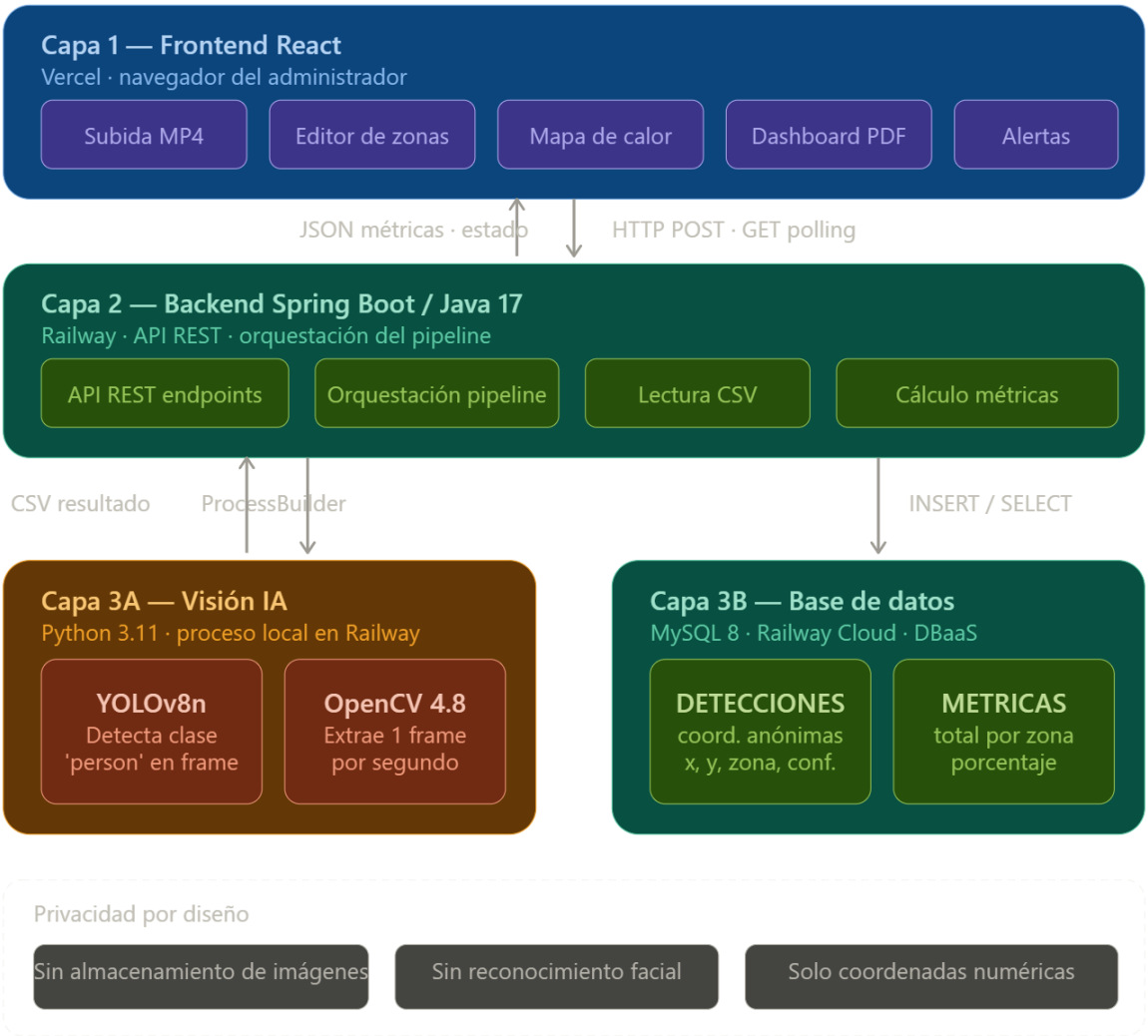
5.2 Diagrama stack Tecnologico



5.3 Diagrama 2: Arquitectura de 4 capas



5.4 Diagrama 3: Flujo de datos



5.5 Arquitectura del sistema por capas

El sistema FlowSense se organiza en 4 capas técnicas desacopladas:

Capa 1 — Frontend React (Vercel)

Interfaz web que el administrador usa en el navegador. Cuatro módulos: (1) Subida de video MP4 con barra de progreso y polling de estado, (2) Editor de zonas sobre el plano del recinto con drag & drop, (3) Mapa de calor con heatmap.js, (4) Dashboard comparativo de métricas por zona con recharts.

Capa 2 — Backend Spring Boot / Java 17 (Railway)

Servidor que orquesta todos los procesos. Recibe el video de React, lo almacena, lanza el script Python mediante ProcessBuilder, espera el resultado, lee el CSV generado, inserta en MySQL, calcula métricas por zona y sirve los datos al frontend mediante API REST.

Capa 3A — Visión IA: Python + YOLOv8 (proceso en Railway)

Script detector.py: abre video con OpenCV, extrae 1 frame por segundo, pasa cada frame por YOLOv8n (umbral configurable, default 0.45), determina zona de cada detección según coordenadas normalizadas, escribe CSV anónimo: id_video, frame_numero, zona_id, x_centro_norm, y_centro_norm, confianza. Nunca almacena imágenes de personas.

Capa 3B — Base de datos: MySQL (Railway Cloud)

5 tablas relacionales. RECINTOS y ZONAS definen la estructura espacial del espacio comercial. VIDEOS registra el estado del procesamiento. DETECCIONES almacena las coordenadas anónimas de cada persona detectada. METRICAS contiene los agregados calculados por Spring Boot.

5.6 Flujo de datos paso a paso

Paso	Flujo de datos completo
1	React: administrador sube video MP4 vía formulario. HTTP POST multipart/form-data a Spring Boot /api/videos/upload.
2	Spring Boot: guarda el archivo MP4 en /uploads/{uuid}.mp4. Registra en MySQL VIDEOS con estado='PENDIENTE'. Devuelve {video_id} al frontend.
3	Spring Boot: ejecuta el script Python via ProcessBuilder: python detector.py --video /uploads/{uuid}.mp4 --output /results/{uuid}.csv --fps 1. Actualiza estado='PROCESANDO'.
4	Python detector.py: abre video con cv2.VideoCapture(). Por cada frame (1/segundo): pasa por model.predict(frame, classes=[0], conf=0.45). Determina zona de cada detección según coordenadas normalizadas. Escribe en CSV: id_video, frame_num, zona_id, x_norm, y_norm, confianza.
5	Spring Boot: al terminar Python (waitFor()), lee el CSV con BufferedReader, hace INSERT masivo en MySQL DETECCIONES, calcula METRICAS por zona (total_detecciones, porcentaje_del_total). Actualiza estado='COMPLETADO'.
6	React: polling cada 3s a GET /api/videos/{id}/estado. Cuando recibe 'COMPLETADO', consulta GET /api/videos/{id}/metricas y pasa los puntos a heatmap.js.
7	heatmap.js recibe array de puntos {x, y, value} y genera el gradiente rojo-azul sobre el canvas del plano del recinto. react-konva gestiona las zonas superpuestas.
8	Exportación PDF: jsPDF + html2canvas capturan el componente React (heatmap + métricas) y generan el PDF directamente en el navegador, sin lógica en el servidor.

5.7 Script Python detector.py — lógica central

El script recibe dos argumentos: ruta del video y ruta del CSV de salida. Ejecuta el siguiente proceso:

- 1. cap = cv2.VideoCapture(ruta_video) — abre el video grabado en el recinto real.
- 2. fps = int(cap.get(cv2.CAP_PROP_FPS)) — obtiene frames por segundo del video.
- 3. Bucle: ret, frame = cap.read(). Si frame_numero % fps == 0, procesar (1 frame/segundo). Si no, continuar.
- 4. results = model.predict(frame, classes=[0], conf=0.45, verbose=False) — detecta personas (clase 0 en COCO).
- 5. Para cada detección: boxes = results[0].boxes.xywhn.cpu().numpy() — coordenadas normalizadas [0-1].
- 6. Determina zona: compara (x_centro_norm, y_centro_norm) con el bounding rect de cada zona definida por el admin.
- 7. Escribe en CSV: id_video, frame_numero, zona_id, x_centro, y_centro, confianza.
- 8. Al finalizar todos los frames, cierra el CSV. Spring Boot lo lee e inserta en MySQL.

5.8 Decisiones éticas del sistema

Decisión ética	Implementación y justificación
No identificación facial	YOLOv8 detecta clase 'person' como caja geométrica (bounding box). No usa reconocimiento facial. El sistema nunca sabe quién es la persona, solo su posición espacial en el frame.
Sin almacenamiento de imágenes	Python procesa frames en memoria RAM y los descarta inmediatamente. Solo persisten coordenadas numéricas (x, y) en MySQL. Ningún archivo de imagen con personas queda almacenado.

Decisión ética	Implementación y justificación
Datos completamente anónimos	Un registro de DETECCIONES contiene: frame_numero=342, zona_id=2, x_centro_norm=0.47, y_centro_norm=0.61, confianza=0.82. Es imposible reconstruir identidad o apariencia de ninguna persona.
Cumplimiento legal Chile	Compatible con la Ley 19.628 de datos personales y la Ley 21.719 de Protección de Datos (vigente 2025). Al no capturar biometría, no requiere consentimiento individual.
Aviso en el recinto	En despliegue real, el administrador debe informar mediante señalética que el recinto usa análisis de flujo anónimo. El proyecto incluye plantilla de aviso en la documentación de entrega.

5.9 Desafíos técnicos y mitigaciones

Desafío técnico	Probabilidad	Mitigación
Instalación YOLOv8 con conflictos de dependencias	Alta	Usar entorno virtual desde día 1: python -m venv venv. Fijar versiones exactas en requirements.txt. Probar instalación completa en Sprint 1.
Acceso a recinto real para grabar video demo	Alta	Coordinar en semana 1. Opciones: galería comercial del barrio, feria de fin de semana, centro comercial. Ángulo desde escaleras o segundo piso.
ProcessBuilder / Runtime.exec falla en Windows	Media	Probar en el SO del equipo en Sprint 1. Alternativa: Python escribe CSV local, Spring Boot tiene un endpoint GET /api/process/{video_id} que lee el CSV manualmente.
Video con ángulo no óptimo para detección	Media	Probar YOLOv8 en un frame antes del Sprint 2. Si la detección es baja, grabar en otro ángulo o recinto con pasillo más visible.
Incompatibilidad de versiones entre librerías React	Media	Fijar versiones exactas en package.json desde Sprint 1: recharts@2.x, react-konva@18.x, heatmap.js@2.x. Probar la combinación completa antes de avanzar al Sprint 2.
Alcance demasiado grande para horas disponibles	Alta	MVP mínimo presentable = subir video + detección Python + puntos en canvas. Zonas, métricas y colores del heatmap son incrementales. El core básico es evaluable desde Sprint 2.

5.10 Criterios de éxito del MVP

El sistema se considera exitoso si cumple todos los siguientes criterios:

- 1. Pipeline completo en < 5 min: subir video de 15 minutos y obtener heatmap en menos de 5 minutos de procesamiento.
- 2. Zonas configurables: el administrador dibuja y guarda mínimo 3 zonas en menos de 5 minutos sin instrucciones.
- 3. Dashboard legible: persona sin conocimiento técnico entiende el reporte comparativo en menos de 2 minutos.
- 4. Exportación PDF funcional: el administrador descarga el reporte PDF con métricas y heatmap en un clic.
- 5. Alertas de zona fría visibles: el sistema indica visualmente las zonas con baja actividad relativa.
- 6. Validación real: el administrador o un representativo confirma que el heatmap refleja la distribución de tráfico percibida en el recinto.

6. Plan de Trabajo

N	Actividad	Descripción	Recursos	Dur.	Responsable	Sprint
1	**Setup entorno Python + YOLOv8**	Crear entorno virtual, instalar ultralytics y opencv, correr primera detección en frame de un video real.	Python, pip, venv	S1	Int. 2	Sprint 1
2	**Modelo MySQL y tablas**	Diseño y creación de las 5 tablas: RECINTOS, ZONAS, VIDEOS, DETECCIONES, METRICAS en MySQL.	MySQL Workbench	S1	Int. 3	Sprint 1
3	**Prototipo UX Figma**	Wireframes de: pantalla subida video, herramienta zonas, vista heatmap, dashboard métricas.	Figma	S1-S2	Int. 1	Sprint 1
4	**Grabación video demo en recinto real**	Coordinar acceso a mall, galería o feria. Grabar pasillo desde ángulo elevado, mínimo 15 minutos en horario de alta afluencia.	Celular, recinto real	S1	Int. 2	Sprint 1
5	**Script detector.py completo**	Leer video con OpenCV, extraer 1 frame/segundo, detectar personas con YOLOv8n, determinar zona por coordenadas, escribir CSV.	Python, ultralytics, OpenCV	S2-S3	Int. 2	Sprint 2
6	**API Spring Boot — subida de video**	Endpoint POST /api/videos/upload que recibe MP4, guarda en disco y registra en MySQL con estado PENDIENTE.	Spring Boot, MySQL	S3	Int. 2	Sprint 2
7	**Orquestación Python desde Spring Boot**	Spring Boot ejecuta detector.py vía ProcessBuilder, espera resultado, lee CSV e inserta en MySQL DETECCIONES.	Spring Boot, Java ProcessBuilder	S3-S4	Int. 2	Sprint 2
8	**Frontend React — subida de video**	Formulario React para subir video con barra de progreso y polling de estado cada 3 segundos.	React, Fetch API	S3-S4	Int. 1	Sprint 2
9	**Herramienta de zonas con react-konva**	Integrar react-konva para que el admin dibuje, mueva y	React, react-konva	S4-S5	Int. 1	Sprint 3

N	Actividad	Descripción	Recursos	Dur.	Responsable	Sprint
		redimensione rectángulos de zonas sobre el plano del recinto con drag & drop.				
10	**Mapa de calor con heatmap.js**	Integrar heatmap.js sobre el plano del recinto. Recibe puntos {x, y, value} desde MySQL y genera el gradiente rojo-azul.	React, heatmap.js, Spring Boot	S5	Int. 1	Sprint 3
11	**Dashboard métricas con recharts**	Mostrar total de detecciones por zona, porcentajes y zona de mayor/menor afluencia. Gráfico de barras comparativo con recharts.	React, recharts, TanStack Table	S6	Int. 1	Sprint 3
12	**Exportación PDF con jsPDF**	html2canvas captura el heatmap + métricas. jsPDF lo inserta en PDF descargable desde el navegador.	jsPDF, html2canvas	S7	Int. 3	Sprint 4
13	**Alertas de zona fría con react-toastify**	Calcular zona con menor actividad relativa. Mostrar toast con react-toastify y badge visual sobre la zona en el mapa.	React, react-toastify	S7	Int. 1	Sprint 4
14	**Gestión multi-recinto**	Selector de recinto en el panel de administración. Cada recinto mantiene sus propias zonas, videos y métricas.	React, Spring Boot, MySQL	S7-S8	Int. 2	Sprint 4
15	**Configuración umbral de confianza**	Control deslizante en UI que pasa el umbral (default 0.45) como parámetro al script detector.py.	React, Spring Boot	S7-S8	Int. 2	Sprint 4
16	**Pruebas de integración E2E**	Probar flujo completo con el video real grabado en Sprint 1. Verificar todos los endpoints.	Spring Boot, React, MySQL	S7-S8	Int. 3	Sprint 4
17	**Documentación técnica final**	README con instrucciones de instalación, variables de entorno, arquitectura y guía de uso.	GitHub, Markdown	S8-S9	Int. 3	Sprint 5

N	Actividad	Descripción	Recursos	Dur.	Responsable	Sprint
18	**Video demo y presentación**	Grabar demo de 3 minutos mostrando el flujo completo con el video real del recinto. Preparar slides de presentación.	OBS Studio, Canva	S9	Todos	Sprint 5

7. Historias de Usuario — Product Backlog

7.1 Qué son las historias de usuario en este proyecto

Las historias de usuario describen las funcionalidades del sistema desde la perspectiva del administrador del espacio comercial. Cada historia define QUÉ necesita el usuario, sus criterios de aceptación medibles y la evidencia que demuestra que fue completada. En conjunto forman el Product Backlog de Scrum.

En FlowSense se definen 10 historias de usuario que cubren el 100% del alcance del MVP. Cada historia está asignada a un sprint específico, tiene un responsable del equipo y está vinculada directamente a una o más evidencias del punto 8. Esta vinculación permite al docente verificar la trazabilidad completa: desde la necesidad del administrador hasta la prueba de que fue implementada.

7.2 Product Backlog

Código	Historia de usuario (Administrador del espacio comercial)	Sprint	Prioridad	Estado	Evidencia vinculada
HU-01	Como administrador, quiero subir un video MP4 de mi recinto para que el sistema lo procese automáticamente y genere métricas de tráfico por zona.	Sprint 2	Alta	Pendiente	EV-01, EV-04
HU-02	Como administrador, quiero definir las zonas de mi recinto sobre el plano para obtener métricas separadas por sector (entrada, pasillo A, zona premium, etc.).	Sprint 3	Alta	Pendiente	EV-01, EV-02
HU-03	Como administrador, quiero ver un mapa de calor sobre el plano de mi recinto para identificar visualmente qué zonas tienen mayor afluencia de público.	Sprint 3	Alta	Pendiente	EV-01, EV-04
HU-04	Como administrador, quiero ver cuántas personas pasaron por cada zona para fundamentar con datos objetivos las diferencias de precio de arriendo entre sectores.	Sprint 3	Alta	Pendiente	EV-01, EV-04
HU-05	Como administrador, quiero consultar el historial de análisis anteriores de mi recinto para comparar períodos y detectar cambios en los patrones de circulación.	Sprint 4	Media	Pendiente	EV-01, EV-04
HU-06	Como administrador, quiero exportar las métricas del análisis como reporte PDF para compartirlo con mis arrendatarios como respaldo objetivo del precio cobrado.	Sprint 4	Media	Pendiente	EV-01, EV-04
HU-07	Como administrador, quiero recibir una alerta visual cuando una zona tenga baja actividad relativa, para detectar pasillos problemáticos y tomar decisiones de rediseño.	Sprint 4	Media	Pendiente	EV-01, EV-04
HU-08	Como administrador, quiero gestionar múltiples recintos (mall, galería, feria) desde una sola cuenta para centralizar el análisis de distintos espacios comerciales.	Sprint 4	Media	Pendiente	EV-01, EV-04

Código	Historia de usuario (Administrador del espacio comercial)	Sprint	Prioridad	Estado	Evidencia vinculada
HU-09	Como administrador, quiero configurar el umbral de confianza mínimo del detector para ajustar la sensibilidad según la calidad del video de mi recinto.	Sprint 4	Baja	Pendiente	EV-01, EV-04
HU-10	Como integrante del equipo, quiero que el repositorio GitHub tenga estructura clara y README con instrucciones de instalación para que cualquiera pueda reproducir el sistema.	Sprint 5	Baja	Pendiente	EV-04

Leyenda de prioridad: Alta = bloqueante para el MVP | Media = importante pero no bloqueante | Baja = mejora de calidad.

8. Evidencias del Proyecto

8.1 Cómo se relacionan evidencias e historias de usuario

Cada evidencia valida una o más historias de usuario. La tabla del punto 7 (Product Backlog) y la tabla del punto 8 (Evidencias) están diseñadas para leerse en conjunto: la columna 'Evidencia vinculada' de cada historia en el punto 7 remite directamente a los códigos EV-XX del punto 8, cerrando el ciclo de trazabilidad. Esta relación garantiza que el proyecto cumple con los tres resultados de aprendizaje del ramo:

- RA1 (necesidad real): validado por EV-01 que documenta la problemática y el contexto de mercado de espacios comerciales chilenos.
- RA2 (solución innovadora con ética): validado por EV-01, EV-02, EV-03 y EV-04 que muestran el diseño, la arquitectura y el código con decisiones éticas documentadas.
- RA3 (validación de la solución): validado por EV-04 y EV-05, que demuestran el sistema funcionando de extremo a extremo con video real de un recinto comercial.

8.2 Tabla de evidencias

Cód.	Nombre de evidencia	Descripción y contenido	HU que valida	Formato / Link
EV-01	**Documento de registro del proyecto (este documento) Entrega semana 4**	Documento completo con los 10 puntos: antecedentes del equipo, descripción del proyecto, contexto con tabla de homologación e Ishikawa, metodología Scrum, descripción técnica con flujo de datos, plan de trabajo, historias de usuario, evidencias y planificación semestral.	HU-01 a HU-10	*.docx | GitHub /docs/registro.docx*
EV-02	**Prototipo UX/UI en Figma Entrega semana 4**	Prototipo interactivo con las 4 pantallas del sistema: (1) Subida de video con barra de progreso, (2) Herramienta de definición de zonas sobre plano del recinto, (3) Vista del mapa de calor con leyenda de colores, (4) Dashboard de métricas comparativas por zona con gráfico de barras.	HU-02, HU-03, HU-04	*Figma link | GitHub /docs/prototipo_link.txt*
EV-03	**Modelo Entidad-Relación (MER) Entrega semana 4**	Diagrama entidad-relación completo de MySQL con las 5 tablas: RECINTOS, ZONAS, VIDEOS, DETECCIONES y METRICAS. Incluye: claves primarias, claves foráneas, tipos de datos de cada campo y cardinalidad de las relaciones entre tablas.	Soporte de BD para todas las HU	*.pdf | GitHub /database/MER.pdf*
EV-04	**Código fuente + README técnico Entrega continua — Sprint a Sprint**	Repositorio GitHub completo con: script Python detector.py, API Spring Boot con endpoints REST, frontend React con canvas heatmap, scripts SQL MySQL, archivo requirements.txt con versiones fijas, .gitignore configurado y README con	HU-01 a HU-10	*GitHub repo | link público en docs*

Cód.	Nombre de evidencia	Descripción y contenido	HU que valida	Formato / Link
		instrucciones de instalación y ejecución paso a paso.		
EV-05	**Informe final + video demo Entrega semana 17-18**	Documento de cierre que incluye: resumen ejecutivo, resultados obtenidos, comparación entre lo planificado y lo ejecutado por sprint, lecciones aprendidas, limitaciones del sistema y recomendaciones de mejora. Acompañado de video demo de 3 minutos mostrando el flujo completo con video real del recinto: subida de video → procesamiento → mapa de calor → métricas → reporte PDF.	Todas las HU	*.pdf + YouTube | GitHub /docs*

8.3 Criterios generales de todas las evidencias

Criterio	Descripción y forma de verificación
Formato de evidencias	Todas las evidencias deben entregarse en los formatos indicados: .docx o .pdf para documentos, link público para el repositorio GitHub, link para el prototipo Figma y link de YouTube para el video demo.
Trazabilidad	Cada evidencia referencia explícitamente los códigos de historias de usuario (HU-XX) que valida. El docente puede verificar la trazabilidad completa en la tabla cruzada de esta sección.
Legibilidad	Todos los documentos deben estar redactados en español, con fuente legible (mínimo 11pt), sin errores ortográficos graves y estructurados según los formatos establecidos por el ramo.
Accesibilidad	El repositorio GitHub debe ser público. El prototipo Figma debe tener permisos de 'view only' para cualquier persona con el link.
Actualización continua	La evidencia EV-04 (código fuente) se actualiza en cada sprint. El docente puede revisar el historial de commits en GitHub para verificar la actividad continua del equipo a lo largo del semestre.

9. Planificación Semestral — 9 semanas

X = semana activa. Los Sprints duran 2 semanas cada uno (Sprint 5 dura 1 semana de cierre).

Actividad	S1	S2	S3	S4	S5	S6	S7	S8	S9
Sprint 1: Setup + YOLOv8 + MySQL + Figma + Video demo en recinto real	X	X							
Sprint 2: Pipeline Python + Spring Boot API			X	X	X				
Sprint 3: Frontend React + Heatmap + Zonas					X	X	X		
Sprint 4: Integración + Admin features + Pruebas E2E							X	X	X
Sprint 5: Documentación + Demo + Presentación								X	X
Daily Standups (todo el semestre)	X	X	X	X	X	X	X	X	X
Sprint Reviews (cada 2 semanas)		X		X		X		X	X
Entrega hito docente (EP1)	X			X					

10. Otros Antecedentes

10.1 Factibilidad técnica

El proyecto es factible con el stack propuesto y las horas disponibles. La clave es la separación de responsabilidades: el integrante 2 domina Python y Spring Boot de forma independiente, el integrante 1 trabaja React en paralelo, y el integrante 3 tiene MySQL listo desde el Sprint 1. Los tres pueden avanzar simultáneamente desde la semana 2 sin bloquearse.

El riesgo principal del Sprint 1 es doble: la instalación de YOLOv8 y la grabación del video demo en un recinto real. Ambos deben completarse en la semana 1. Si el video se graba en una galería o feria de acceso público, el permiso se gestiona directamente con el administrador del espacio. Si la instalación de YOLOv8 presenta conflictos, el entorno virtual aislado desde el día 1 los contiene sin afectar el resto del stack.

10.2 Modelo de negocio (visión futura del producto)

Plan	Precio	Incluye
Freemium	**$0 / mes**	3 análisis de video/mes, hasta 4 zonas por recinto, mapa de calor básico. (Captación de usuarios.)
Plan Administrador	**$29.990 CLP / mes**	Análisis ilimitados, histórico de 6 meses, comparativo entre períodos, alertas de zona fría, exportación PDF, 1 recinto.
Plan Multi-recinto	**$59.990 CLP / mes**	Todo el plan Administrador + hasta 5 recintos, reporte consolidado por recinto, API para integración con sistemas propios.

10.3 Proceso de negocio afectado

El proceso de negocio que FlowSense transforma es la fijación de precios de arriendo y la negociación con arrendatarios en espacios comerciales con múltiples locales o stands.

Proceso actual (sin FlowSense): el administrador observa el recinto intuitivamente → decide diferencias de precio de forma subjetiva → no puede justificar técnicamente ante arrendatarios que cuestionan el precio → negocia a la baja sin evidencia → no mide el impacto de cambios en la distribución de locales.

Con FlowSense: administrador graba video del recinto → sube el video a la plataforma → define zonas sobre el plano → obtiene datos objetivos de tráfico por zona → toma decisiones de precio basadas en evidencia → mide el impacto en el siguiente período → comparte el reporte PDF con arrendatarios. El ciclo de fijación de arriendos pasa de ser intuitivo a ser basado en datos.

10.4 Instrucciones de grabación del video demo

El video de demostración debe grabarse en un recinto real (mall, galería o feria) durante el Sprint 1. Instrucciones:

- Coordinar acceso: en galerías pequeñas y ferias, hablar directamente con el administrador. En malls, buscar un punto de vista desde escaleras o segundo piso sin necesidad de permiso especial.
- Ángulo: elevado o cenital (simula cámara de seguridad). Las personas deben verse claramente separadas y caminando.
- Duración mínima recomendada: 15 a 30 minutos en horario de alta afluencia (hora de almuerzo o tarde de fin de semana).
- Resolución mínima: HD (1280x720). Un celular moderno es suficiente.
- Zonas a definir: identificar al menos 3 zonas claramente diferenciadas en el recinto (por ejemplo: entrada/acceso, pasillo central, pasillo lateral o zona premium).

- Guardar el archivo como recinto_demo.mp4 en el directorio /test_videos del repositorio (agregar a .gitignore para no subir archivos grandes a GitHub).
- Privacidad: el ángulo cenital o semi-cenital garantiza que los rostros no sean identificables, haciendo el video compatible con el sistema de detección anónima por diseño.